Bench setup - TRDJ600RZ simulator (NUCLEO-G431KB)

Operating sheet for the debug session with `printf` on ITM and capture with the LA1010 logic analyzer (or OWON oscilloscope). Reference state: 2026-10-02 build, `Code=10628`, 0 Errors / 0 Warnings, TIM2 configured in external clock mode 1 from TIM3.

---

1. Measurement chain

```
   COMMAND                  NUCLEO-G431KB                          instruments
   -------                  -------------                          ---------
   optical knob             PB6  (TIM4_CH1) KNOB_A  --+
   Bourns ENA1J --> 5 V --->PB7  (TIM4_CH2) KNOB_B  --+--> LA1010 CH5/CH6
(optional)
                     GND --->GND

                            PA6  (TIM3_CH1) A  ----------> LA1010 CH1
                            PA4  (TIM3_CH2) B  ----------> LA1010 CH2
                            PB0  (TIM3_CH3) -  ----------> (no signal)
                            PA0  (TIM2_CH1) Z > LA1010 CH0  (capture trigger)
                            PB8  (LED)      anchor       > LA1010 CH4  (printf
instant)
                            PB3  (SWO)  ------------------> ST-LINK -> printf
Viewer
                            GND  ------------------------> common GND
```

Wiring rules:

- **Common GND is mandatory** between board, analyzer and knob.
- **Do not use A4 / PA5** and **do not drive A5**: they are pads shared with the knob and with output A. The full detail (SB2/SB3 jumpers, 3.3 V and 5 V I/O classes, accesses on the connector) is in **section 4**.
- The debug pins (PA13/PA14 SWD, PB3 SWO) do not interfere with the measurement pins.
- Oscilloscope probe: short ground on the nearest GND, otherwise the 20 ns edges get dirty and the phase delays become unreadable.

---

2. Keil uVision - debug and SWV trace

Values already present in the project (verified after the CubeMX regeneration):

Where	Item	Value
Options -> Debug	Debugger	**ST-Link Debugger** (`pMon = STLink\ST-LINKIII-KEIL_SWO.dll`)
Debug -> Settings -> Debug	Port / Max Clock	**SW / 10 MHz** (`DbgClock = 10000000`)
Debug -> Settings -> Trace	Core Clock	**170 MHz** (`-TC170000000`)
Debug -> Settings -> Trace	Trace Enable	**checked** (`-TIE1`)
Debug -> Settings -> Trace	ITM Stimulus Port **0**	**checked** (`-TIP1`)
Utilities -> Settings	Reset and Run	active, if you want the program to start after the download
View -> Analysis Windows	**Debug (printf) Viewer**	receive window of the ITM channel
View	**Periodic Window Update**	active (`periodic=1`): refreshes watch and viewer while the program runs

Operating notes:

- With no debugger connected the ITM is off: `ITM_SendChar()` returns at once and `printf` writes nothing. This is the expected behaviour, not a fault.
- With the core in **halt** the timers keep counting (they are not frozen): the signal stays present on the pins and the analyzer measurement is still valid with the program stopped.
- Useful registers in the Watch window (they are the "true" reference, read from the registers): `TIM3->CNT` , `TIM3->ARR` , `TIM3->CCR1` , `TIM3->CCR2` , `TIM3->CCR3` , `TIM3->PSC` , `TIM2->CNT` , `TIM2->SMCR` , `TIM4->CNT` . Quick Z check on TIM2: **`TIM2->SMCR` with the three low bits equal to 111 (External Clock Mode 1)**, that is `(TIM2->SMCR & 0x7) == 0x7` . On TIM3 the matching check is **`TIM3->CR2` with MMS = 010 (Update)**, the trigger output that clocks TIM2: the Z pulse exists only if both hold.

---

3. Control knob (KNOB_A / KNOB_B)

The knob is the only input of the system: the speed and the direction of the simulated rotation come from it, so it must be verified together with the generator.

Item	Value
Encoder	optical knob **Bourns ENA1J-B28-L00100L**, 100 pulses/revolution, 5 V supply
Pin A	**PB6 = KNOB_A = TIM4_CH1**, AF2, No pull, output speed Low
Pin B	**PB7 = KNOB_B = TIM4_CH2**, AF2, No pull, output speed Low
Levels	5 V outputs (VOH >= 4 V, VOL <= 0.8 V); PB6 and PB7 are **5 V tolerant** (FT_c / FT_f classes), so they connect directly, with no level shifter
Encoder supply	5 V from the board (CN3 pin 4, green LED LD4 5V_PWR on), max current draw 26 mA; common GND
TIM4 mode	encoder mode **TI1 and TI2** (`TIM_ENCODERMODE_TI12`), PSC 0, ARR 65535
Input filter	`IC1Filter = IC2Filter = 8` -> sampling at f_DTS/8 with 6 samples: the level must stay stable for about **0.28 us** to be accepted
Counts	400 counts per revolution in x4 reading (100 pulses/revolution x 4)
Reading	**signed** difference between two reads of the 16-bit counter (the counter wraps at 65535 and is never written absolutely)
Scaling	each count is worth **14** position units (`KNOB_DELTA_SCALE`)
Position	from **-16800** to **+16800**: the sign selects the direction, the magnitude the speed
Dead zone	none: at the limits the position saturates but the counter keeps being read, so on the way back the movement resumes at once
Apply rate	at most every **50 ms** (`KNOB_APPLY_MS`): every application resets the TIM3 counter, so while rotating the signal phase jumps and that is normal
TIM4 start	`HAL_TIM_Encoder_Start()` in `KnobInput_Init()` : in encoder mode the peripheral is configured but the counter stays still until it is enabled
Start position	at power-up the position is **0** whatever the physical knob position: the rotation starts at the first detent

What to watch in debug:

- `TIM4->CNT` must change when the knob is rotated: with the knob still it is stable, and one full turn moves it forward (or backward) by about **400**.
- In one knob turn the speed rises by about **517 rpm** (400 x 14 = 5600 position units over a full scale of 16800 that covers 50..1600 rpm).
- On the analyzer (CH5 = PB6 and CH6 = PB7, if you also want to check the input): two waves in quadrature with A leading by 90 deg in clockwise rotation and **100 periods per revolution** on each channel. At hand speed the frequency is a few hundred Hz, much lower than the generated signal: to see them together a long window is needed (at the top of the range, 1 s at 10 MS/s contains the 600 A/B periods of one revolution together with the knob edges).
- If the knob controls the speed **in the opposite way**, the cause is the swap of A and B: it is fixed by inverting the two wires, or by letting the sign of the position select the direction (`SpindleSim_SetDirection()`).

---

4. Jumpers SB2/SB3, I/O classes and shared pads

Two pairs of MCU pins share the same Arduino connector pad, and the choice is made by a jumper soldered **on the bottom side** of the board. It concerns the two knob inputs (PB6, PB7) and the simulator output A (PA6).

4.1 How the shared pads work

The connector pad is **a single one**, and the jumper decides which of the two MCU pins is connected to it. It is not a link between the two MCU pins: it is a selection.

```
PA5 / PB7   -->  pad A4 = CN3 pin 8
PA6 / PA15  -->  pad A5 = CN3 pin 7
```

The official board pinout drawing (UM2397 Rev 2, Figure 12, page 21) shows both alternatives on the two pads: `A4` as **`PA5/PB7`** and `A5` as **`PA6/PA15`** . The same figure also gives the pin mapping we use: **D4 = PB7**, **D5 = PA15**, **D6 = PB6**, **CN3 pin 7 = A5**, **CN3 pin 8 = A4**.

4.1.1 SB2 and SB3 table

From the official jumper table (UM2397 Rev 2, Table 9, section 6.7):

Jumper	State	Effect
SB2	**ON**	PB7 connected to **CN3 pin 8** (I2C SDA, A4). PB7 no longer supports D4 and **PA5 must be configured as a floating input**
SB2	OFF	CN3 pin 8 used as A4 without I2C; **CN4 pin 7** available as D4, with PA5 on pad A4
SB3	**ON**	PA15 connected to **CN3 pin 7** (I2C SCL, A5). PA15 no longer supports D5 and **PA6 must be configured as a floating input**
SB3	OFF	CN3 pin 7 used as A5 without I2C; **CN4 pin 8** available as D5, with PA6 on pad A5

UM2397 explicitly refers to this table for the limits on A4 and A5 and states that the selection is made **by remapping, without changing the hardware**: it is an electrical selection, not a logical one, so the firmware does not see it.

4.1.2 Configuration of this unit: SB2 = ON, SB3 = ON

Actual state, verified with the ohmmeter (0 ohm resistors fitted, not desoldered):

Jumper	Function	State	Consequence on the project
SB2	PB7 to CN3 pin 8 (A4)	**ON**	KNOB_B can be reached **only on A4**; pin D4 is no longer the knob
SB3	PA15 to CN3 pin 7 (A5)	**ON**	**output A on PA6 ends up on pad A5**; PA15 is absent from the project, so the second pin of the pad is free
SB4	T_SWO on PB3	**ON**	needed for the ITM channel of `printf`
SB6 / SB7	user LED LD2 on PB3 or on PB8	SB6 **OFF**, SB7 **ON**	board default: LED on PB8, as in the firmware. **SB6 and SB7 have no relation at all with A4, A5, PA5 or PA15**: they select only the LED
SB9 / SB10	24 MHz crystal X2 on PF0/PF1	**ON**	the firmware uses HSE and runs at 170 MHz, so the crystal is active

Jumper	Function	State	Consequence on the project
SB1 / SB12	USART2 (PA2/PA3) to the VCP	as on the board	do not touch
SB16	AGND connected to GND	**reserved: do not modify**	

There are 16 jumpers, all on the bottom side. The manual states the factory state in bold, but this board **is not in the factory state** (SB9/SB10 ON are needed for the crystal).

4.1.3 Why PA5 and PA15 must stay floating

With SB2/SB3 ON, the alternative pin of the shared pad **is not free: it is on the same node**. It must be left as a **floating input** and never configured as an output, otherwise it contends for the pad.

In the `.ioc` file of the project the condition is already met: **PA5 and PA15 do not appear at all** (no `PA5.*` / `PA15.*` in the file), so they are in the reset state, that is an analog input at high impedance.

PA6 on the other hand is configured and active: `PA6.Signal=S_TIM3_CH1`, labelled `ENCODER_A`. With SB3 ON that pin is electrically on pad A5, so **A5 carries our output A** and must not be connected to anything that drives it.

The I/O classes of the pins involved are not symmetrical, and the difference matters:

Pin	Class	Maximum voltage	Notes
PB6	**FT_c**	5.5 V	5 V tolerant
PB7	**FT_f**	7.3 V	5 V tolerant
PA5	**TT_a**	**4.0 V**	**not** 5 V tolerant
PA6	**TT_a**	**4.0 V**	**not** 5 V tolerant
PA15	**FT_f**	7.3 V	5 V tolerant

Overvoltage. On pad A4, PB7 is `FT_f` (5 V allowed) but PA5 is `TT_a` (absolute maximum 4.0 V). A 5 V signal brought to A4 is therefore allowed for PB7 and **outside the absolute maxima for PA5**. The analog mode **does not disable the protection diodes**: current injection occurs even with PA5 left floating, and the high level on the node can turn out

limited with rounded edges. This is also why the board rule applies even when PA5 is left as an input: that note assumes a 3.3 V I2C bus, not a 5 V signal.

On A5 with SB3 ON the same asymmetry is more marked: PA6 is `TT_a` and PA15 is `FT_f`.

Shared pad. Whatever is connected to A4 (jumper, shield, another output) ends up on the PB7 node, that is on knob input B. A high-impedance probe is fine; a driving source is not. The same holds symmetrically for A5, which carries our output A.

4.2 Access to the signals on the connector (with SB2 = ON and SB3 = ON)

Signal	MCU pin	Connector pad	Class
A	PA6	**A5** (CN3 pin 7)	`TT_a`
B	PA4	**A3** (CN3 pin 9)	`TT_a`
Z (1 pulse/revolution)	PA0	**A0** (CN4 pin 4)	`TT_a`
PB0	PB0	D3 (CN4 pin 6)	`TT_a` - not used (TIM3_CH3 never enabled)
KNOB_A	PB6	**D6** (CN4 pin 9)	`FT_c`
KNOB_B	PB7	**A4** (CN3 pin 8)	`FT_f`
SWO (printf)	PB3	**D13** (CN4 pin 15)	-

Two consequences to keep in mind when wiring:

- pin **D4 (CN4 pin 7) cannot be used for the knob**: with SB2 ON PB7 has been moved to A4;
- pin **D5 (CN4 pin 8) carries PA6**, that is our output A, with SB3 ON.

For the knob the conclusion is binding in the opposite sense to what one might assume: TIM4_CH2 exists **only** on PB7 in this package (LQFP32), so PB7 must be used, but **there is no single valid pad**: with SB2 ON it is A4, with SB2 OFF it is D4. Whatever the state of the jumper, PB7 can be reached.

4.3 How to verify the state of SB2 (and SB3)

- **Visual**: jumper on the bottom side of the board, identifiable by the `SB2` silkscreen (and `SB3`).

- **Electrical**, board off and knob connected but not powered, using the wire **B** of the encoder as the reference (it is the PB7 node):
  - wire B <-> **A4** (CN3 pin 8): about 0 ohm means **SB2 ON**;
  - wire B <-> **D4** (CN4 pin 7): about 0 ohm means **SB2 OFF**. For SB3 the measurement is instead **A5 <-> D5**: continuity present means SB3 OFF.
- **Symptom on the analyzer** with SB2 ON and the knob powered at 5 V: on wire B the high level turns out limited and the edges rounded (the PA5 diode conducts) instead of the full 5 V level with clean edges. This is the signature of the injection on PA5.

4.4 Adopted configuration and resulting constraints

SB2 = ON and SB3 = ON are the current configuration and must be kept. There is no reason to desolder: the knob is fully usable, with KNOB_B on A4 and KNOB_A on D6.

The resulting constraints are on **PA5** and on **PA15**, not on the knob pins:

- **PA5 must stay in** `Reset_State` (floating analog input) and must not be configured as an output, otherwise it contends with PB7 on pad A4. PA5 is also `TT_a` (absolute maximum 4.0 V): an encoder powered at 5 V produces injection in its protection diode even when floating. If the high level on A4 turns out limited and the edges rounded, this is the cause.
- **PA15 must stay in** `Reset_State`, for the same reason with respect to PA6 on pad A5.
- **PA6 is an active output** (`ENCODER_A`, TIM3_CH1): the pad A5 that carries it must not be connected to anything that drives it.

4.5 Operating rule on levels

The 5 V signals (the knob) connect only to PB6 and PB7. On the `TT_a` pins (PA0, PA4, PA6, PB0, that is pads A0, A3, A5, D3) only 3.3 V levels may arrive: outputs A, B, Z and the trigger to the ELS controller, or the measurement instruments.

This is also why the trigger was moved from PA5 to **PA0**: PA5 is `TT_a` and with SB2 ON it is on the same node as the knob powered at 5 V.

---

5. `printf` instrumentation (to be inserted in the loop)

In `main()`, the declaration before the `while (1)` and the block inside the loop after `KnobInput_Poll()`:

```
/* ===== Bench instrumentation: active only with the debugger connected ===== */
/* 250 ms rate, aligned with the SpindleSim_MeasureRpm() window: each line
   blocks the loop for hundreds of us while the ITM FIFO drains, so it must
   never be placed in the hot path without a rate limit. */
static uint32_t t_log = 0U;

if ((HAL_GetTick() - t_log) >= 250U) {
    long   set10 = (long)(SpindleSim_GetRpm() * 10.0f + 0.5f);  /* tenths of rpm
*/
    float  m     = SpindleSim_MeasureRpm();                     /* -1 if stopped
*/
    long   mis10 = (long)(m * 10.0f + 0.5f);                    /* -10 = stopped
*/

    t_log = HAL_GetTick();

    /* Anchor: the pulse on PB8 bounds the blocking time of the printf and
       allows the printed line to be correlated with a precise instant of the
trace */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_SET);
    printf("set10=%ld mis10=%ld pwm=%lu ARR=%lu CCR=%lu/%lu/%lu knob=%ld\r\n",
           set10, mis10,
           (unsigned long)SpindleSim_GetPwmHz(),
           (unsigned long)TIM3->ARR,
           (unsigned long)TIM3->CCR1, (unsigned long)TIM3->CCR2, (unsigned
long)TIM3->CCR3,
           (long)KnobInput_GetPosition());
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_RESET);
}
```

How to read it:

- `set10` / `mis10` are speeds in **tenths of rpm** (`set10 = 5999` means 599.9 rpm); `knob` is the position in units. `mis10 = -10` means rotation stopped, estimate not available. Printing floats as scaled integers avoids pulling in the library floating-point formatter (`%f`) and halves the blocking time of the line.
- The three `CCR` at 600 rpm must read `3541 / 10625 / 7083` with `ARR = 14166` .
- **The `printf` disturbs the measurement you are making**: during the block the loop does not call `SpindleSim_MeasureRpm()` , so `mis10` reads about 0.2-0.4 % less than `set10` . The pulse width on PB8 is exactly that blocking time: it is a measurement, not an estimate. If `mis10` falls far below `set10` , there is other work in the loop besides printing.

---

6. LA1010 - channels, sampling, trigger

Setting	Recommended value
CH0	**PA0 / A0** = Z (1 pulse/revolution) and capture trigger
CH1	**PA6 / A5** = A
CH2	**PA4 / A3** = B
CH3	free (PB0 generates no signals)
CH4	**PB8** = printf anchor
CH5 / CH6	**PB6 / D6** and **PB7 / A4** = knob (only if you want to check the input)
Threshold	1.4 V (3.3 V logic; on the knob the levels are 5 V and the same threshold remains valid)
Sampling rate	**10 MS/s** (100 ns/sample); 25 MS/s if more resolution is needed
Window	1.2 s (one revolution at 50 rpm); at 1600 rpm 100 ms is enough
Trigger	**rising edge on CH0**, normal/single mode
Measurements to read	frequency and period of A; delay A->B; A->Z phase (T/8 before the rising edge of A); **count of A edges between two Z pulses**

At 1600 rpm the A->B delay is 15.6 us: at 10 MS/s that is ~156 samples, sufficient; below 10 MS/s the phase comparison becomes imprecise. If the analyzer memory is not enough for 1 s, reduce the window while keeping the sampling rate: resolution matters more than duration. To measure knob and generator together keep in mind that the two signals differ by two or three orders of magnitude in frequency (hundreds of Hz against tens of kHz).

---

7. Expected values on the generator

	50 rpm	1400 rpm	1600 rpm
A and B: frequency	500 Hz	14 000 Hz	16 000 Hz
A and B: period	2000 us	71.4 us	62.5 us
Delay A->B (T/4)	500 us	17.9 us	15.6 us

	50 rpm	1400 rpm	1600 rpm
Z: frequency (1 pulse/revolution = rpm/60)	0.83 Hz	23.3 Hz	26.7 Hz
A->Z phase (Z starts T/8 before the rising edge of A)	-250 us	-8.93 us	-7.81 us
Duty of A and B	50 %	50 %	50 %
A periods in one window	**600**	**600**	**600**
Z: pulse width (2 events = 1 A/B period)	2 ms	71.4 us	62.5 us

Z is one pulse per revolution: frequency rpm/60 (0.83 Hz at 50 rpm, 26.7 Hz at 1600 rpm) and a pulse 2 update events of TIM3 wide, that is one A/B period (167 us at 600 rpm). The pulse starts from the reset of the TIM2 counter, which happens on the update event of TIM3: with respect to the rising edge of A it therefore starts T/8 earlier. PB0 generates no signal.

---

8. Knob positions for known speeds

The position is linear between `SPINDLE_SIM_RPM_MIN` (50 rpm) and `SPINDLE_SIM_RPM_MAX` (1600 rpm) over +/-16800, with a gain of 14 per count: **one count ~ 1.29 rpm**, **one knob turn ~ 517 rpm** and **three turns cover the whole range**.

Position	Speed
+/-16 800	1600 rpm (maximum)
+/-14 000	1342 rpm
+/-11 200	1083 rpm
+/-8 400	825 rpm
+/-5 600 (1 turn)	567 rpm
+/-2 800 (1/2 turn)	308 rpm
+/-1 400 (1/4 turn)	179 rpm
0	50 rpm (minimum)

The speed is applied at most every **50 ms** and every application resets the counter of TIM3: it is normal for the signal phase to jump while the knob is rotating.

---

9. Procedure

1. Check the jumpers with the board off: **SB2 = ON and SB3 = ON** are the adopted configuration (section 4.1.2) and must be kept. Check that **PA5 and PA15 stay floating** in CubeMX (section 4.1.3) and verify SB4 (SWO on PB3) and SB6/SB7 (LED on PB8).
2. Connect the knob (5 V, GND, KNOB_A on PB6, KNOB_B on PB7) and the analyzer channels (CH0 = PA0, CH1 = PA6, CH2 = PA4, CH3 = PB0, CH4 = PB8), with common GND; power the board over USB (ST-LINK).
3. In uVision: download, open **Debug (printf) Viewer** and the **Watch** window with the registers.
4. Start the debug session, leave the program in run, enable **Periodic Window Update**.
5. Check the input: while rotating the knob `TIM4->CNT` must change and the speed must update in steps of 50 ms (it is the rate limit, not a defect). If you measure wire B, the high level must be the full 5 V with clean edges: a limited and rounded level means that PA5 is not floating or is injecting (section 4.1.3).
6. Arm the analyzer in normal/single mode with trigger on the **rising edge of CH0**.
7. Rotate the knob up to the wanted speed (table in section 8): the rotation starts at the first detent, the first trigger pulse arrives after about one revolution. Bringing the knob back to **0** the rotation stops and `SpindleSim_Stop()` calls `HAL_TIM_PWM_Stop()` on TIM2, channel 1: **PA0 is driven back to low level** instead of staying at the reached level. If you step again the rotation restarts without the trigger staying frozen high. The TIM2 counter restarts from the parking value (2, not 0), so **the first revolution after every restart stays shorter by 2 steps out of 1200** (about 167 us at 600 rpm): it is the same semantics as before the introduction of the hook and must be kept in mind when comparing the width of the first captured window with the following ones.
8. Check on the analyzer: Z period, pulse width, **600 A edges** between two Z pulses, A->B delay = T/4, A->Z phase = T/8 before the rising edge of A, duty of A and B 50 %. If you also connected CH5/CH6, check the quadrature of the knob: two waves shifted by 90 deg, A leading when rotating clockwise.
9. Compare with the `printf` line: `pwm` and `ARR` / `CCR` must match what you measure, and `knob` must match the position you have set.
10. The pulse on PB8 tells how long the print stopped the loop: if `mis10` is lower than expected, that is the cause and the pulse width is the proof.

---

10. Quick diagnostics

Symptom	Probable cause	Verification
PA0 emits a train at ~141.7 kHz	`TIM2->SMCR` with SMS = 000: slave mode not active, TIM2 counts on the internal clock	`TIM2->SMCR & 0x7` must be `0x7`
Z (PA0) silent, A present	`HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1)` missing, or TIM2 not in External Clock Mode 1	`SpindleSim_Start()`, `TIM2->SMCR` with the three low bits = 111
A, B and Z all silent	rotation stopped (knob at 0: `SpindleSim_Stop()`)	rotate the knob
Z (PA0) stays **high** with the knob still	`SpindleSim_Stop()` has not disabled the TIM2 channel	`SpindleSim.c:462` : `HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_1)`
The knob does not change the speed	encoder not powered, or TIM4 not started	5 V and GND on the encoder; `TIM4->CNT` still; `HAL_TIM_Encoder_Start()` in `KnobInput_Init()`
The speed changes in the opposite direction	KNOB_A and KNOB_B swapped	invert the two wires or check `SpindleSim_SetDirection()`
The speed does not update while you rotate	50 ms rate limit	it is the expected behaviour
The position stays at +/-16800	full scale reached (3 knob turns)	bring the knob back: there is no dead zone
Knob high level limited and edges rounded	SB2 ON: the 5 V wave reaches PA5 (TT_a) and the protection diode conducts	section 4.1.3: PA5 must be floating; **do not** move SB2
A5 connected to a driving source	with SB3 ON pad A5 is shared with our output A on PA6	section 4.2: leave A5 free, do not configure PA15

Symptom	Probable cause	Verification
5 V on A0, A3, A5 or D3	those pins are TT_a (absolute maximum 4.0 V)	5 V signals go only to PB6/PB7 (section 4.5)
Frequencies double with respect to `pwm`	APB1 prescaler different from 1: the driver reads `PCLK1` , the timer is `2 x PCLK1`	`RCC.APB1Freq_Value` = `APB1TimFreq_Value` = 170 MHz
`pwm` different from 5999 at 600 rpm	ARR not reprogrammed or system clock different from 170 MHz	`TIM3->ARR` = 14166, `TIM3->PSC` = 0
Z absent on PB0	expected behaviour: TIM3_CH3 is not enabled	see section 7
Z low for ~0.6 period at every knob rotation	phase realignment at every geometry application (50 ms rate limit)	expected behaviour
`printf Viewer` empty	ITM off, wrong Core Clock, or debugger not connected	Trace: 170 MHz, ITM and Port 0 active
`mis10` far below `set10`	starved loop (printing too often or other work)	pulse width on PB8
A/B phase inverted on the generator	simulated rotation direction	`SpindleSim_SetDirection()` / sign of the knob position

---

11. References in the current state

File	Role
`...\Drivers\Spindle_sim\SpindleSim.c/.h`	generation of A and B (TIM3) and of the Z index (TIM2), speed estimate
`...\Core\Src\main.c`	`MX_TIM2_Init()` (Z: External Clock Mode 1 on ITR2, ARR 1199, Pulse 2), `MX_TIM3_Init()` (A/B, TRGO = Update, CH3 configured but never enabled), `MX_TIM4_Init()` (knob)

File	Role
`...\Drivers\Knob_input\KnobInput.c/.h`	knob: position, direction, 50 ms rate limit
`...\Drivers\Utils\retarget_io.c`	retarget of `printf` -> ITM (SWO); it is linked to the target (`printf` is called from `Core\Src\main.c:118` , and the file is in the `Drivers/Utils` group)